Kubernetes Networking – Tutorial Report

Objective 1: Kubernetes Cluster Setup

Step 1: VM Creation in OpenStack

Created three VMs on OpenStack using the Debian-10-OpenStack image:

- kubemaster (192.168.56.74/24)
- kubenode1 (192.168.56.11/24)
- kubenode2 (192.168.56.12/24)

All VMs were connected to the internal-network-1 and their hostnames and IPs were manually added to /etc/hosts.

Host VM

```
stack@nvo-sneha:~$ cat /etc/hosts | grep kube
172.24.4.250 kubenode2
172.24.4.231 kubenode1
172.24.4.90 kubemaster
stack@nvo-sneha:~$
```

kubemaster

```
debian@kubemaster:~$ cat /etc/hosts | grep kube
127.0.1.1 kubemaster.novalocal kubemaster
172.24.4.231 kubenode1
172.24.4.250 kubenode2
```

kubenode1

```
debian@kubenode1:~$ cat /etc/hosts | grep kube
127.0.1.1 kubenode1.novalocal kubenode1
172.24.4.90 kubemaster
172.24.4.250 kubenode2
```

kubenode2

```
debian@kubenode2:~$ cat /etc/hosts | grep kube
127.0.1.1 kubenode2.novalocal kubenode2
172.24.4.231 kubenode1
172.24.4.90 kubemaster
```

Screenshots: /etc/hosts on each VM

kubemaster

```
debian@kubemaster:~$ python3 ping_all.py
+------------+----------------+-------------+
|    Host    |       IP       | Ping Status |
+------------+----------------+-------------+
| kubemaster | 192.168.56.74  |   Success   |
| kubenode1  | 192.168.56.221 |   Success   |
| kubenode2  | 192.168.56.135 |   Success   |
|  host-vm   |   172.24.4.1   |   Success   |
+------------+----------------+-------------+
debian@kubemaster:~$
```

kubenode1

```
debian@kubenode1:~$ python3 ping_all.py
+------------+----------------+-------------+
|    Host    |       IP       | Ping Status |
+------------+----------------+-------------+
| kubemaster | 192.168.56.74  |   Success   |
| kubenode1  | 192.168.56.221 |   Success   |
| kubenode2  | 192.168.56.135 |   Success   |
|  host-vm   |   172.24.4.1   |   Success   |
+------------+----------------+-------------+
debian@kubenode1:~$
```

kubenode2

```
debian@kubenode2:~$ python3 ping_all.py
+------------+----------------+-------------+
|    Host    |       IP       | Ping Status |
+------------+----------------+-------------+
| kubemaster | 192.168.56.74  |   Success   |
| kubenode1  | 192.168.56.221 |   Success   |
| kubenode2  | 192.168.56.135 |   Success   |
|  host-vm   |   172.24.4.1   |   Success   |
+------------+----------------+-------------+
debian@kubenode2:~$
```

Screenshots: Ping results

Step 2: System Configuration

Disabled swap and enabled IP forwarding on all nodes.

```
swapoff -a
echo 'net.ipv4.ip_forward = 1' >> /etc/sysctl.conf
```

Step 3: Install containerd

Installed containerd and configured `SystemdCgroup = true` in */etc/containerd/config.toml*

```
sudo apt update
sudo apt install -y containerd
sudo mkdir -p /etc/containerd
containerd config default | sudo tee /etc/containerd/config.toml
```

Then restarted the containerd service

```
sudo systemctl restart containerd
sudo systemctl enable containerd
```

Step 4: Install Kubernetes Components

Added signed-by APT keyring and installed kubelet, kubeadm, and kubectl using the Kubernetes v1.32 stable repository.

```
sudo apt update
sudo apt install -y apt-transport-https ca-certificates curl gpg

curl -fsSL https://pkgs.k8s.io/core:/stable:/v1.32/deb/Release.key | \
  gpg --dearmor | sudo tee /etc/apt/keyrings/kubernetes-apt-keyring.gpg >
/dev/null

echo "deb [signed-by=/etc/apt/keyrings/kubernetes-apt-keyring.gpg] \
https://pkgs.k8s.io/core:/stable:/v1.32/deb/ /" | \
sudo tee /etc/apt/sources.list.d/kubernetes.list

sudo apt update
```

```
sudo apt install -y kubelet kubeadm kubectl
sudo apt-mark hold kubelet kubeadm kubectl
```

Step 5: Initialize the Control Plane

On kubemaster:

```
sudo kubeadm init --pod-network-cidr=192.168.0.0/16 \
  --apiserver-advertise-address=192.168.56.74
```

Configured kubectl for the regular user by copying admin.conf to *~/.kube/config*

```
mkdir -p $HOME/.kube
sudo cp /etc/kubernetes/admin.conf $HOME/.kube/config
sudo chown $(id -u):$(id -g) $HOME/.kube/config
```

```
Your Kubernetes control-plane has initialized successfully!

To start using your cluster, you need to run the following as a regular user:

  mkdir -p $HOME/.kube
  sudo cp -i /etc/kubernetes/admin.conf $HOME/.kube/config
  sudo chown $(id -u):$(id -g) $HOME/.kube/config

Alternatively, if you are the root user, you can run:

  export KUBECONFIG=/etc/kubernetes/admin.conf

You should now deploy a pod network to the cluster.
Run "kubectl apply -f [podnetwork].yaml" with one of the options listed at:
  https://kubernetes.io/docs/concepts/cluster-administration/addons/

Then you can join any number of worker nodes by running the following on each as root:

kubeadm join 192.168.56.74:6443 --token 4yaic9.co62datz6hduynni \
        --discovery-token-ca-cert-hash sha256:dc5b969f57dc69cc23de34fc8b852ed91cf3e0132935273c2823d0c5b224997f
debian@kubemaster:~$
```

```
debian@kubemaster:~$ kubectl cluster-info
Kubernetes control plane is running at https://192.168.56.74:6443
CoreDNS is running at https://192.168.56.74:6443/api/v1/namespaces/kube-system/services/kube-dns:dns/proxy

To further debug and diagnose cluster problems, use 'kubectl cluster-info dump'.
debian@kubemaster:~$
```

Screenshot: kubeadm init output and kubectl cluster-info

Step 6: Install Calico CNI

Installed Calico networking plugin using v3.26 manifest.

```
kubectl apply -f
https://raw.githubusercontent.com/projectcalico/calico/v3.26.1/manifests/ca
lico.yaml
```

```
debian@kubemaster:~$ kubectl get pods --all-namespaces
NAMESPACE     NAME                                       READY   STATUS    RESTARTS   AGE
kube-system   calico-kube-controllers-689744956f-jlffv   1/1     Running   0          94s
kube-system   calico-node-q47tl                          1/1     Running   0          95s
kube-system   coredns-668d6bf9bc-b94gz                   1/1     Running   0          5m7s
kube-system   coredns-668d6bf9bc-cd6fj                   1/1     Running   0          5m8s
kube-system   etcd-kubemaster                            1/1     Running   0          5m21s
kube-system   kube-apiserver-kubemaster                  1/1     Running   0          5m21s
kube-system   kube-controller-manager-kubemaster         1/1     Running   0          5m21s
kube-system   kube-proxy-64t8c                           1/1     Running   0          5m8s
kube-system   kube-scheduler-kubemaster                  1/1     Running   0          5m21s
debian@kubemaster:~$
```

Screenshot: kubectl get pods --all-namespaces

Step 7: Join Worker Nodes

Joined both worker nodes using the kubeadm join command from the master output.

On kubenode1 and kubenode2:

```
sudo kubeadm join 192.168.56.74:6443 --token <token> \
  --discovery-token-ca-cert-hash sha256:<hash>
```

```
debian@kubemaster:~$ kubectl get nodes
NAME         STATUS   ROLES           AGE     VERSION
kubemaster   Ready    control-plane   9m25s   v1.32.3
kubenode1    Ready    <none>          2m15s   v1.32.3
kubenode2    Ready    <none>          2m19s   v1.32.3
debian@kubemaster:~$
```

Screenshot: kubectl get nodes

Objective 2: SDN Controller Deployment

Step 1: Deploy a Single Ryu Pod

Created a single pod using osrg/ryu container image with ryu.app.simple_switch. Used ryu-pod.yaml to apply the configuration.

```
kubectl apply -f ryu-pod.yaml
```

```
debian@kubemaster:~$ kubectl get pods
NAME              READY   STATUS    RESTARTS   AGE
ryu-controller    1/1     Running   0          10h
[debian@kubemaster:~$
```

Screenshot: kubectl get pods

Step 2: Create a ReplicaSet

Created a ReplicaSet using ryu-replicaset.yaml to maintain 3 instances of the Ryu controller.

```
kubectl apply -f ryu-replicaset.yaml
kubectl get pods -o wide
```

```
[debian@kubemaster:~$ kubectl get pods -o wide
NAME                  READY   STATUS    RESTARTS   AGE   IP               NODE        NOMINATED NODE   READINESS GATES
ryu-controller        1/1     Running   0          10h   192.168.35.65    kubenode2   <none>           <none>
ryu-replicaset-krwxr  1/1     Running   0          45s   192.168.35.66    kubenode2   <none>           <none>
ryu-replicaset-xl8rb  1/1     Running   0          45s   192.168.205.193  kubenode1   <none>           <none>
debian@kubemaster:~$
```

Screenshot: kubectl get pods -o wide

Step 3: Expose Ryu Controller via NodePort

Exposed the Ryu controller using a NodePort service on port 30633. Used ryu-nodeport.yaml for this setup.

```
kubectl apply -f ryu-nodeport.yaml
kubectl get services
```

```
[debian@kubemaster:~$ kubectl get services
NAME          TYPE        CLUSTER-IP      EXTERNAL-IP   PORT(S)          AGE
kubernetes    ClusterIP   10.96.0.1       <none>        443/TCP          17h
ryu-service   NodePort    10.107.196.23   <none>        6633:30633/TCP   8s
[debian@kubemaster:~$
```

Screenshot: kubectl get services

Step 4: Connect Mininet to Controller

Ran Mininet from a remote machine and connected to the Ryu controller using:

```
sudo mn --controller=remote,ip=172.24.4.90,port=30633
--topo=tree,depth=2,fanout=2
```

```
[sneha@nvo-sneha:~$ sudo mn --controller=remote,ip=172.24.4.90,port=30633 --topo=tree,depth=2,fanout=2
[sudo] password for sneha:
/usr/local/bin/mn:4: DeprecationWarning: pkg_resources is deprecated as an API. See https://setuptools.pypa.io/en/latest/pkg_resources.html
  __import__('pkg_resources').run_script('mininet==2.3.1b4', 'mn')
*** Creating network
*** Adding controller
*** Adding hosts:
h1 h2 h3 h4
*** Adding switches:
s1 s2 s3
*** Adding links:
(s1, s2) (s1, s3) (s2, h1) (s2, h2) (s3, h3) (s3, h4)
*** Configuring hosts
h1 h2 h3 h4
*** Starting controller
c0
*** Starting 3 switches
s1 s2 s3 ...
*** Starting CLI:
mininet> pingall
*** Ping: testing ping reachability
h1 -> h2 h3 h4
h2 -> h1 h3 h4
h3 -> h1 h2 h4
h4 -> h1 h2 h3
*** Results: 0% dropped (12/12 received)
mininet>
```

Screenshot: Mininet pingall output (0% packet loss)

Step 5: Failover Test

Deleted one Ryu pod manually using `kubectl delete pod` . Verified that ReplicaSet automatically created a new one. In the screenshot below, the three pods were named 2vhrl, krwxr, and xl8rb. After deleting 2vhrl, a new pod named vz6cp was automatically created to replace it.

```
[debian@kubemaster:~$ kubectl get pods
NAME                   READY   STATUS    RESTARTS   AGE
ryu-replicaset-2vhrl   1/1     Running   0          6m25s
ryu-replicaset-krwxr   1/1     Running   0          9m43s
ryu-replicaset-xl8rb   1/1     Running   0          9m43s
[debian@kubemaster:~$
[debian@kubemaster:~$
[debian@kubemaster:~$ kubectl delete pod ryu-replicaset-2vhrl
pod "ryu-replicaset-2vhrl" deleted



debian@kubemaster:~$
debian@kubemaster:~$
debian@kubemaster:~$
[debian@kubemaster:~$ kubectl get pods
NAME                   READY   STATUS    RESTARTS   AGE
ryu-replicaset-krwxr   1/1     Running   0          10m
ryu-replicaset-vz6cp   1/1     Running   0          39s
ryu-replicaset-xl8rb   1/1     Running   0          10m
```

Screenshot: Old pod gone, new pod started
